



Jinja2 Masterclass

Domine a arte de criar templates HTML dinâmicos com Python

Desenvolvimento Web Moderno

| Exibindo Dados e Dicionários

Variáveis Simples

Utilize chaves duplas para renderizar valores passados pelo backend.

```
Nome: {{ nome }}  
Idade: {{ idade }}
```

Estruturas Complexas

Acesse atributos de objetos ou chaves de dicionários com facilidade.

```
{{ aluno.nome }}  
{{ aluno["nota"] }}
```

| Estruturas de Repetição (Loop For)

O bloco for permite iterar sobre listas, tuplas ou dicionários diretamente no HTML.

- ✓ Sintaxe limpa: {% for item in lista %}
- ✓ Obrigatório fechar com {% endfor %}
- ✓ Preserva a indentação visual do HTML

```
{% for aluno in alunos %}

    {{ aluno.nome }}

{% endfor %}
    {{ aluno.nome }}
```

| Variáveis do Objeto Loop

Dentro de um loop, o Jinja2 oferece o objeto loop com propriedades úteis:

Propriedade	Descrição	Exemplo Prático
loop.index	Iteração atual (inicia em 1)	{{ loop.index }} - Nome
loop.index0	Iteração atual (inicia em 0)	ID: {{ loop.index0 }}
loop.first	Retorna True se for o 1o item	{% if loop.first %}...{% endif %}
loop.last	Retorna True se for o último item	Separadores, vírgulas finais

| Lógica Condicional (If / Else)

Condição Simples

Controle a visibilidade de elementos com base em estados.

```
{% if nota >= 7 %}  
    Aprovado  
{% else %}  
    Reprovado  
{% endif %}
```

Condições Múltiplas

Use elif para fluxos de decisão mais complexos.

```
{% if nota >= 9 %}  
    Excelente!  
{% elif nota >= 7 %}  
    Aprovado  
{% else %}  
    Reprovado  
{% endif %}
```

ARQUITETURA

Herança de Templates

A herança permite reutilizar o layout base (Header, Footer, Navbar) em todas as páginas, mudando apenas o miolo.

| O Poder do extends e block

Base (Pai)

Define as "âncoras" onde o conteúdo será injetado.

```
{% block content %}  
{% endblock %}
```

Filho (Página)

Preenche os blocos definidos no arquivo pai.

```
{% extends 'base.html' %}  
  
{% block content %}  
    Minha Página  
{% endblock %}
```

| Modularização com include

O `{% include %}` é ideal para componentes reutilizáveis em qualquer lugar, como menus e rodapés.

- ✓ Mantém os arquivos menores e legíveis
- ✓ Facilita a manutenção global
- ✓ Pode ser usado dentro de loops

```
{% include "header.html" %}  
...  
{% include "footer.html" %}
```


| Filtros de Transformação



Strings

upper / lower

replace("a", "@")



Coleções

length (tamanho)

sort (ordenar)



Formatação

join(", ")

title (Iniciais)

Uso: {{ nome | upper }} ou {{ lista | length }}

| Operadores Matemáticos



Básicos

`{{ 1 + 2 }}` : Soma

`{{ 5 * 2 }}` : Mult



Avançados

`{{ 10 / 2 }}` : Divisão

`{{ 10 % 3 }}` : Módulo



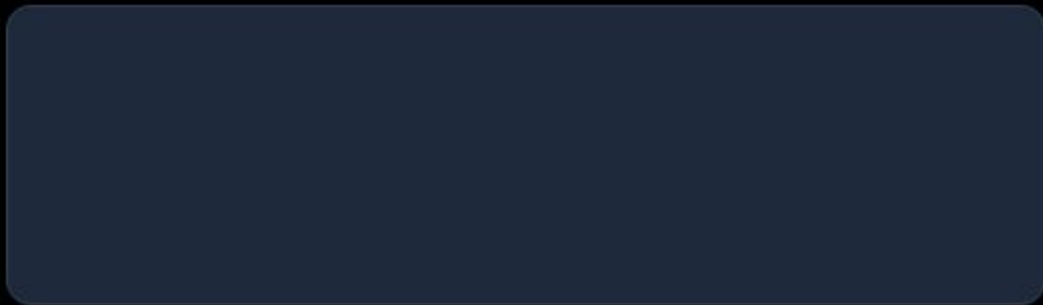
Potência

`{{ 2 ** 3 }}` : Exponenciação

(2 elevado a 3)

| Static Files e url_for

Em frameworks como Flask e FastAPI, gerenciamos CSS, JS e imagens via `url_for` para garantir caminhos absolutos corretos.



Dica: Sempre coloque seus arquivos na pasta `/static` do projeto.

Dúvidas?

Obrigado pela atenção!



Use {# comentário #}
para notas invisíveis no
HTML final.



Explore Macros para
criar funções
reutilizáveis em seus
templates.

| Image Sources



Thumbnail for

<https://storage.ghost.io/c/49/77/4977795e8-85fb-4775-848e-ee5126597212/content/images/size/w600/format/webp/2024/09/pc-media--42-.png>

Source: www.packetcoders.io

www.packetcoders.io



Thumbnail

https://miro.medium.com/l*7Vq-CAUby3nt0UkF4GKhUg.png

Source: medium.com

for



medium.com

Thumbnail

<https://cdn.sanity.io/images/bj34pdlbp/migration/3dd7ff672c7b36fc06bf0eda3ae537c0a61a7276-674x421.png?w=3840&q=75&fit=clip&auto=format>

Source: blog.replit.com

for

blog.replit.com
